Reference: Engine API

Reference: Engine API

This page describes every public function and data structure in `engine/`. It states facts only. For explanations of why the engine works this way, see `docs/explanation/tiers-explained.md` and `docs/white-paper.md`.

`engine/fj_wager.py` — Tier 1 (guarantee-based)

```
legal_max_wager(score: int) -> int
```

Returns the largest legal Final Jeopardy wager for a given score. This equals the score itself. A player cannot wager more money than they have.

```
worst_case_ceiling(opponent_score: int) -> int
```

Returns the highest score an opponent could reach: their current score, doubled. This assumes the opponent wagers everything and answers correctly.

```
analyze(scores: List[int], player_index: int) -> FJRecommendation
```

Computes the Tier 1 wager for one player against all others.

Parameters:

Name	Type	Meaning
`scores`	list of int	Every player's current score, in any order.
`player_index`	int	The position, in `scores`, of the player to analyze.

Returns: an `FJRecommendation` with these fields:

Field	Type	Meaning
`lockout`	bool	`True` if no opponent can catch this player, even if that opponent bets everything and wins.
`min_wager_to_guarantee_win`	int or None	The smallest wager that beats every opponent's worst case outright. `None` if no wager can guarantee a win.
`max_legal_wager`	int	The largest legal wager (the player's own score).
`lock_tie_wager`	int or None	A wager that locks in at least a tie, when one exists.

Field	Type	Meaning
`binding_opponent_index`	int	The opponent whose worst case sets the requirement — the single largest threat, not a combination of opponents.

`analyze_all(scores: List[int]) -> List[FJRecommendation]`

Runs `analyze()` for every player and returns one `FJRecommendation` per player, in the same order as `scores`.

`engine/fj_bestresponse.py` — Tier 2 (confidence-weighted)

`OpponentModel`

A data structure describing one opponent for a Tier 2 analysis.

Field	Type	Meaning
`score`	int	The opponent's current score.
`wager_dist`	list of (int, float) pairs	Each possible wager for this opponent, paired with its probability. Probabilities must sum to 1.0.
`p_correct`	float	The probability this opponent answers correctly, if they wager. Defaults to 0.5.

`default_opponent_wager_model(scores, opponent_index) -> WagerDist`

Builds an illustrative wager distribution for one opponent, anchored on that opponent's own Tier 1 breakpoints (bet nothing, their cover wager if one exists, their lock-tie wager if one exists, bet everything), with equal probability across whichever of those apply. This is a placeholder, not a value fitted to real data — see `docs/adr/0002-opponent-model-assumptions.md`.

`expected_win_probability(self_score, self_wager, p_self_correct, opponents) -> float`

Returns the exact probability that the analyzed player ties or beats every opponent, for one specific wager. Computed by exact enumeration over the discrete opponent models supplied, not by simulation.

`analyze(scores, player_index, p_self_correct, opponent_model_fn=default_opponent_wager_model, opponent_correctness=0.5, wager_step=100) -> FJBestResponse`

Searches candidate wagers and returns the one with the best win probability.

Parameters:

Name	Type	Meaning
`scores`	list of int	Every player's current score.
`player_index`	int	The position of the player to analyze.
`p_self_correct`	float	The analyzed player's own confidence, from 0.0 to 1.0.

Name	Type	Meaning
`opponent_model_fn`	function	Builds each opponent's wager distribution. Defaults to `default_opponent_wager_model`.
`opponent_correctness`	float	The assumed correctness probability for every opponent. Defaults to 0.5.
`wager_step`	int	The gap, in dollars, between candidate wagers tested. Defaults to 100.

Returns: an `FJBestResponse` with these fields:

Field	Type	Meaning
`recommended_wager`	int	The wager with the best win probability. Among tied wagers, the one with the higher strict-win (no-tie) probability is chosen, then the smallest wager.
`win_probability`	float	The win probability at the recommended wager, ties counted as wins.
`wager_win_probabilities`	list of (int, float)	Every candidate wager tested, paired with its win probability.
`tier1_cover_wager`	int or None	The Tier 1 guarantee wager, for comparison.

`engine/dd_wager.py` — Daily Double engine

`legal_max_dd_wager(score: int, round_number: int) -> int`

Returns the largest legal Daily Double wager: the greater of the current score or the round's clue-value cap ($1,000 in Single Jeopardy, $2,000 in Double Jeopardy).

`analyze(scores, player_index, round_number, p_correct, config=None, wager_step=100, trials_per_wager=3000, z_score=1.0, seed=None) -> DDRecommendation`

Runs a Monte Carlo simulation of the rest of the game for each candidate wager, and returns the wager with the best simulated win rate.

Parameters:

Name	Type	Meaning
`scores`	list of int	Every player's current score.
`player_index`	int	The position of the player to analyze.
`round_number`	int	`1` for Single Jeopardy, `2` for Double Jeopardy.
`p_correct`	float	The analyzed player's confidence in this Daily Double clue.

Name	Type	Meaning
`config`	`RolloutConfig` or None	Describes the rest of the game — remaining clue values, remaining Daily Doubles. See `engine/simulate.py`.
`wager_step`	int	The gap, in dollars, between candidate wagers tested. Defaults to 100.
`trials_per_wager`	int	Simulated trials run per candidate wager. Defaults to 3000. Raise this for a tighter estimate.
`z_score`	float	How many combined standard errors define the "plateau" of statistically similar wagers. Defaults to 1.0.
`seed`	int or None	A fixed random seed. The same seed always produces the same result.

Returns: a `DDRecommendation` with these fields:

Field	Type	Meaning
`recommended_wager`	int	The smallest wager on the plateau (see below).
`win_probability`	float	The estimated win probability at the best wager.
`wager_win_probabilities`	list of (int, float)	Every candidate wager tested, paired with its estimated win probability.
`standard_errors`	list of (int, float)	Each candidate wager's standard error, `sqrt(p(1-p)/trials)`.
`plateau`	list of int	Every wager whose estimate is statistically indistinguishable from the best one.
`notes`	list of str	Plain-language caveats about this specific result.

`engine/simulate.py` — shared rollout model

`RolloutConfig`

Describes the assumptions used to simulate the rest of a game, after a Daily Double decision.

Field	Type	Meaning
`remaining_clue_values`	list of int	The dollar value of each regular clue still on the board.
`remaining_dd_count`	int	How many more Daily Doubles are still hidden.

Field	Type	Meaning
`regular_clue_success_rate`	float	Assumed chance any given regular clue is answered correctly. Defaults to 0.6.
`fj_confidence`	float	Assumed Final Jeopardy confidence used inside the rollout. Defaults to 0.5.
`future_dd_wager_fraction`	float	Assumed fraction of score wagered on a future, not-yet-found Daily Double. Defaults to 0.5.
`future_dd_success_rate`	float	Assumed chance a future Daily Double is answered correctly. Defaults to 0.6.

Every constant here is a named, documented placeholder, not a value fitted to real historical data. See `RISK_REGISTER.md` risk #5 and #7.

`rollout(scores, player_index, config, rng) -> bool`

Simulates the rest of the game forward once, from the given scores, and returns `True` if the analyzed player wins (ties count as wins) at the end.